Neurocomputing 537 (2023) 187–197

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Event-triggered near-optimal tracking control based on adaptive dynamic programming for discrete-time systems ☆

Ziyang Wang [a], Joonhyup Lee [b], Qinglai Wei [c,*], Anting Zhang [b]

[a] Department of Mechanical and Electrical Engineering, Xilingol Vocational College, Xilinhot 026000, China
[b] Department of Automation, Tsinghua University, Beijing 100084, China
[c] The State Key Laboratory of Management and Control for Complex Systems Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China

ARTICLE INFO

Article history:
Received 17 April 2022
Revised 7 September 2022
Accepted 28 March 2023
Available online 31 March 2023

Keyword:
Optimal control
Adaptive dynamic programming
Event-triggered control
Neural network

ABSTRACT

Frequent state monitoring and controller updates can enhance the precision of tracking control, while simultaneously overburdening the communication network transmission. In this paper, For the purpose of saving communication costs, we propose event-triggered control algorithms for the optimal tracking control problem. First, we reconstruct the discrete-time nonlinear system into a converted system. Then, the adaptive dynamic programming algorithm is employed to find the optimal controller off-line, and the event-triggered scheme is used to reduce the communication costs online. Novel triggering conditions with fewer assumptions are designed to implement the event-triggered scheme. Different from existing works, the event-triggered scheme can be introduced not only into the converted system but also into the actual system, which is more practical because the actual controller is what one can only access in practice. In addition, with the developed algorithms, the tracking error can be proved to be stable at the origin, in other words, the actual system can be guaranteed to track the desired trajectory. Algorithms developed in this paper are implemented by three neural networks, the model network, the action network and the critic network. Finally, examples are presented to verify the effectiveness and rationality of the algorithms.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

With the onset of the big data era, conservation of resource consumption and communication costs is gradually the mainstream demand [1,2]. However, equidistant sampling is always considered in traditional time-triggered control systems, which is a conservative way to guarantee the control performance and sacrifice communication costs [3]. In practice, most physical systems have finite-bandwidth between sensors, controllers, and actuators, such as multi-agent systems and wireless sensor actuator networks [4–6]. To trade off the performance against the costs, the event-triggered control (ETC) method is implemented by aperiodic sampling. The event-triggered controller is executed once the event is triggered [7]. Thus, ETC methods could reduce the traffic of information and save the communication resources [8]. Recently, many results on event-triggered approaches have emerged [9–16], including event-triggered adaptive dynamic programming (ADP) methods [17,18].

As a classical optimal control problem, the optimal tracking control problem [19,20] can be solved by ADP methods efficiently [21–26]. Many existing works [27–29] convert the optimal tracking control problem into an optimal regulation problem firstly, then solve the optimal regulation problem by ADP methods. In this paper, the system of the optimal tracking control problem is called the actual system, and the system of the optimal regulation problem is called the converted system. ADP is a self-learning method based on neural networks with the actor-critic structure [30–33], which has been applied widely in practice, such as micro-grids, solar energy, and unmanned underwater vehicles [34–39]. As data amounts expand, the necessity to economize on communication costs grows, and the event-triggered adaptive dynamic programming (ADP) algorithm appears [40–43].

The event-triggered ADP method consists of two parts: off-line optimization and online control. First, the iterative ADP algorithms are employed to obtain the optimal control policy off-line. Then, we can execute the optimal controller online by the event-triggered scheme. Thus, the event-triggered ADP method is to

This work was supported by the National Key Research and Development Program of China (No. 2019YFB1404904)

* Corresponding author.
E-mail addresses: wangziyang14@163.com (Z. Wang), lijunxie19@mails.tsinghua.edu.cn (J. Lee), qinglai.wei@ia.ac.cn (Q. Wei), zhanganting@tsinghua.edu.cn (A. Zhang).

https://doi.org/10.1016/j.neucom.2023.03.045
0925-2312/© 2023 Elsevier B.V. All rights reserved.

balance the performance and costs based on demand by executing optimal control policy intermittently. The iterative ADP algorithm contains two groups: value iterations (VI) and policy iterations (PI), whose implementation process can be found in [23,33]. Policy iteration has been demonstrated to make systems stable, but requires a known initial admissible control policy. Value iteration does not have the need for this condition, but the stability of the system is not easily guaranteed. Thus, we obtain the optimal control policy based on PI in this paper.

The event-triggered ADP method is an ideal tool to solve discrete-time optimal tracking control problems, which is with high requirement of accuracy and smooth communication. And several achievements have looked into this topic [44–47]. Although many researchers have focused on the event-triggered ADP methods, most are in the continuous-time domain [46]. There are also realistic discrete-time control systems where the burden of communication needs to be reduced, such as scheduling problems, path planning problems, etc. Besides, most of the existing event-triggered optimal controllers designed for discrete-time systems are only applicable to the optimal regulated systems. Even in solving the optimal tracking problem, the optimal tracking problem is first converted to an optimal regulation problem, and then an event-triggered controller is designed for the converted system [44]. Only Lu et al. [45] proposed an event-triggered ADP algorithm for actual controller, however, their algorithm is only guaranteed that the tracking error is UUB, instead of stable at the origin. According to the above, an event-triggered algorithm which accommodates system stability and performance optimality is urgently needed to solve the discrete-time optimal tracking control problems. This inspired our research.

In this paper, two event-triggered ADP algorithms are developed, one applying the event-triggered mechanism to the converted controller and the other applying the event-triggered mechanism to the actual controller. The tracking error can be guaranteed to be stable at the origin, that is, as time tends to infinity, the state of actual system can reach the trajectory being tracked. We employ the ADP method to access the optimal controller offline firstly. Then, to trade off the performance against the communication costs, we update the control input through the event-triggered scheme. The main contributions of this paper are:

1) To solve the optimal tracking control problem with limited communication costs, we develop two event-triggered ADP algorithms. Different from existing works [44,48], the event-triggered scheme can be incorporated both into the actual controller and into the converted controller, which is more practical.
2) Novel triggering conditions are designed for the developed algorithms. Different from existing works [49], we diminish assumptions by inserting a new triggering condition. Thus, the feasibility of the event-triggered ADP algorithm is improved.
3) Different from existing works [45], the stability of the tracking error at the origin is guaranteed based on the developed algorithms, consequently the actual system can be guaranteed to track the desired trajectory accurately.

The remaining contents are structured as follows. The formulation of the optimal tracking control problem, consisting of the system conversion and the optimal regulation problem, is introduced in Section II. In Section III, two event-triggered control algorithms are given, and the asymptotic stability is proved. In Section IV, the neural network implementation is declared. Besides, in Section V, we test these algorithms with two simulations. In Section VI, we conclude this paper.

Notation The transpose of a matrix A is denoted by A^T and The inverse of the matrix is denoted by A^{-1} . $\mathbb{R}$ denotes the set of real numbers. $\mathbb{R}^n$ denotes the n-dimensional Euclidean space. $\|\cdot\|$ is the denotation of Euclidean norm of vectors or matrices. $\{k_j\}_{j=0}^{\infty}$ denotes a set containing $k_0, k_1, k_2, \ldots, k_\infty$.

2. Optimal tracking control problems

Define a desired trajectory [48] to be tracked as

$$\beta_{k+1} = \psi(\beta_k). \tag{1}$$

Suppose the desired trajectory is bounded. Consider the actual system as a discrete-time nonlinear affine system, given by

$$x_{k+1} = f(x_k) + g(x_k)u_k, \tag{2}$$

where $x_k \in \mathbb{R}^n$ is the state of the actual system, and $u_k \in \mathbb{R}^m$ is the actual control. Assume that f and g are differentiable in their argument with $f(0) = 0$, and g is invertible.

The purpose of the tracking control system is that the actual system can track the desired trajectory through manipulating the actual controller u_k. To reach this goal, we reconstruct the optimal tracking control problem to an optimal regulation problem, which can be solved by iterative ADP methods. Consequently, the actual system is transformed into a converted system, and the actual controller is transformed into a converted controller.

2.1. System conversion

The system conversion is illustrated in this section. Define the tracking error as

$$d_k = x_k - \beta_k. \tag{3}$$

where β_k is the desired trajectory. According to [27], the feedforward control input μ_k corresponding to β_k can be obtained by

$$\mu_k = g^{-1}(\beta_k)(\beta_{k+1} - f(\beta_k)), \tag{4}$$

where g^{-1} is the inverse of g. In this paper, v_k is named as converted control, given by

$$v_k = u_k - \mu_k. \tag{5}$$

Substitute (2), (4), and (5) into (3), the converted system can be obtained by

$$d_{k+1} = f(d_k + \beta_k) + g(d_k + \beta_k)(v_k + g^{-1}(\beta_k)(\beta_{k+1} - f(\beta_k))) - \beta_{k+1}, \tag{6}$$

which can be simply written as

$$d_{k+1} = F(d_k, v_k). \tag{7}$$

The structure of the system conversion is shown in Fig. 1. In this paper, the system (7) corresponding to d_k and v_k is named as the converted system. Thus, the tracking control problem for the actual system is transformed into an optimal regulation problem for the converted system. In applications, the actual system can be manipulated directly, while the converted system cannot.

2.2. Optimal regulation problem

Define the utility function for the converted system (7) as

$$U(d_k, v_k) = d_k^{\mathrm{T}} Q d_k + v_k^{\mathrm{T}} R v_k, \tag{8}$$

where $Q \in \mathbb{R}^{n\times n}$ and $R \in \mathbb{R}^{m\times m}$ are positive semi-definite symmetric matrices with appropriate dimensions. The performance index function is expressed as

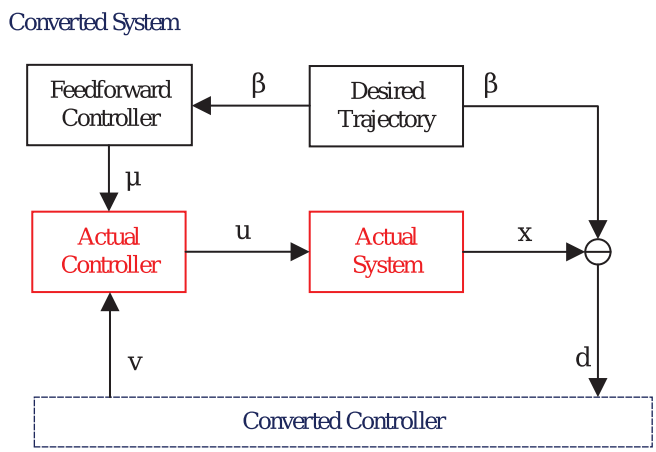

Fig. 1. The structure of the system conversion.

$$J(d_k) = \sum_{j=k}^{\infty} U(d_j, v_j). \tag{9}$$

To make the actual system (2) track the desired trajectory (1), the performance index function should be minimized. On the basis of Bellman optimality principle, the optimal performance index function $J^*(d_k)$ can be given as

$$\begin{aligned} J^*(d_k) &= \min_{v_k}\{U(d_k, v_k) + J^*(d_{k+1})\} \\ &= \min_{v_k}\{U(d_k, v_k) + J^*(F(d_k, v_k))\}. \end{aligned} \tag{10}$$

The optimal control law v_k^* is

$$v_k^* = \arg\min_{v_k}\{U(d_k, v_k) + J^*(F(d_k, v_k))\}. \tag{11}$$

The optimal performance index function and the optimal control law can be obtained by solving (10) and (11), respectively. However, it is difficult to solve these non-analytical equations. Thus, the ADP algorithm is employed to obtain the optimal solution in this paper. Moreover, considering the requirement of timeliness for tracking control systems, we incorporate the event-triggered scheme into the online controlling process, which is described as follows.

3. Event-triggered near-optimal tracking control algorithms and stability analysis

In this section, we present two event-triggered near-optimal tracking control algorithms based on the ADP method. Each algorithm contains two parts, off-line optimization and online control. ADP algorithm is used to obtain the optimal control policy off-line during the optimization process, and we execute the optimal control policy by event-triggered scheme online during the control process. That is, the control input will be updated to the optimal one once the event is triggered. Although the optimal control policy can be obtained by ADP algorithm, the control input is not always the optimal one due to the event-triggered mechanism. Therefore, compared with the time-triggered ADP algorithm, we can reduce communication costs but only obtain a near-optimal performance.

The event triggering mechanism is applied to the controllers. As can be seen from (4), μ_k is obtained relying on the target trajectory and is time-varying. While both the actual controller v_k and the converted controller u_k are variables, input by users. These two controllers can either be real-time updated or intermittently changed, which means that the event triggering mechanisms can be introduced. Apparently, there can be two kinds of event-triggered controllers here, event-triggered converted controller $v_k = v_{k_j}$ and event-triggered actual controller $u_k = u_{k_j}$. Therefore, when the converted controller v_k is event-triggered, since μ_k is time-varying, u_k can be time-varying due to (5). Similarly, when the actual controller u_k is event-triggered, v_k can also be time-varying. Hence, two event-triggered ADP algorithms are proposed in this paper.

3.1. The off-line optimization

The off-line optimization is implemented through the ADP method, which can obtain the optimal controller by iterative training between iterative value function and iterative control policy. Let $V_i(d_k)$ be the iterative value function and $v_i(d_k)$ be the iterative control policy, both of which are implemented by neural networks. The iterative value function is used to approximate the optimal performance index function, satisfying the HJB equation. The iterative converted control policy can be derived from the iterative value function. Specifically, based on the PI algorithm, the iterative value function and the iterative converted control policy are updated by

$$V_i(d_k) = U(d_k, v_i(d_k)) + V_i(F(d_k, v_i(d_k))), \tag{12}$$

$$v_{i+1}(d_k) = \arg\min_{v_i}\{U(d_k, v_i(d_k)) + V_i(F(d_k, v_i(d_k)))\}. \tag{13}$$

According to [24], the control policy will approach the optimal one as the number of iterations i increases to infinity, i.e.,

$$v_k^* = \lim_{i\to\infty} v_i(d_k). \tag{14}$$

3.2. Event-triggered converted controller

In this subsection, we study the first scenario that the converted system is operated with the event-triggered scheme, that is, the converted controller is event-triggered. When the event is triggered, the converted input will be updated and fed into a zero-order holder (ZOH). On the contrary, the system will be controlled by the ZOH.

Define triggering instants as $\{k_j\}_{j=0}^{\infty}$. j represents the number of times that the event has been triggered. Then, d_{k_j} represents the tracking error at triggering instants. Thus, the converted control v_k can be rewritten as $v_k = v_{k_j}, \quad k_j \leqslant k < k_{j+1}$. Hence, the converted system (7) is given by $d_{k+1} = F(d_k, v_{k_j})$. At instant k, define the triggering error as the difference between the state of the previous event trigger instant and the state of the current instant, expressed as

$$e_k = d_{k_j} - d_k, \quad k_j \leqslant k < k_{j+1}, \tag{15}$$

Subsequently, the event triggering conditions designed in this paper are given. The triggering conditions consist of two inequalities, one restricts the triggering error from exceeding the threshold, and the other restricts the prediction state. An event will be triggered when at least one of the inequalities holds.The triggering conditions are originally designed, mainly based on the fact that the system can be kept stable. The triggering conditions are written as follows,

$$\|e_k\|^2 > T_e, \tag{16}$$

$$\|F(d_k, v_k)\| > T_f, \tag{17}$$

where $\|\cdot\|$ is the denotation of 2-norm. T_e and T_f are triggering thresholds equivalent to

$$T_e = (1 - \alpha C^2)\|d_k\|^2/2C^2, \tag{18}$$

$$T_f = C\|d_k\| + C\|e_k\|, \tag{19}$$

where C is a parameter to be determined according to the historical data of the system, $\alpha \geqslant 2$ can be chosen depending on the demand for performance.

When at least one of the triggering conditions (16) and (17) is satisfied, the converted control v_k will be updated, or it will keep constant. The ETC for converted systems (ETCC) is presented in Algorithm 1.

Algorithm 1: ETCC

Initialization
Let $k = 0; h = 0$;
Take the optimal control policy v_k^* as control input.
1: Compute the triggering error e_k according to (15);
2: Compute the threshold according to (18) and (19);
3: **if** at least one of (16) and (17) is satisfied **then**
4: Update the tracking control input $v_k = v_k^*$.
5: Update the tracking error $d_{k+1} = F(d_k, v_k)$.
6: Set $h = h + 1$.
7: Update the ZOH $v_h = v_k$.
8: Set $k = k + 1$ and goto Step 1.
9: **else**
10: Update the tracking error $d_{k+1} = F(d_k, v_h)$.
11: Set $k = k + 1$, and goto Step 1.

The architecture block diagram is shown in Fig. 2. According to Fig. 2 and (5), although the converted controller is updated intermittently, the actual controller is updated continuously with the feedforward controller. Therefore, this swill still lead to continuous communication consumption.

Remark 1. We propose Algorithm 1, mainly to provide such an idea of designing an event-triggered controller. As can be seen, this event-triggered controller only guarantees that the converted controller is event-triggered, i.e., the actual controller is still time-triggered. This is less significant for saving communication resources. Certainly, we are exploring some unique systems where this approach is applicable.

The stability of the converted system determines whether the actual system can track the desired trajectory. Based on the developed ETCC algorithm, the stability analysis of the converted system is presented as follows.

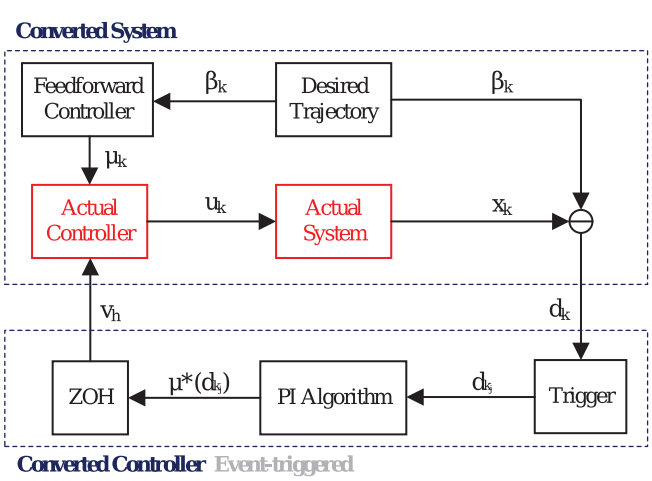

Fig. 2. The event-triggered control algorithm for the converted system.

Theorem 1. *Based on the adaptive dynamic programming algorithm and the event-triggered converted controller, It can be proved that the converted system (7) is stable. Consequently, the actual system (2) can be guaranteed to track the desired trajectory.*

Proof :
Define a Lyapunov function as follows

$$L_k = d_k^T Q d_k + v_k^T R v_k. \tag{20}$$

When neither the triggering condition (16) nor (17) is satisfied, the converted control v_k will not be updated in the period. The first difference of Lyapunov function (20) can be written as

$$\begin{aligned}\Delta L_k = \quad & (d_{k+1}^T Q d_{k+1} - d_k^T Q d_k) + (v_{k+1}^T R v_{k+1} - v_k^T R v_k) \\ & = d_{k+1}^T Q d_{k+1} - d_k^T Q d_k \\ & \leqslant \lambda_q(\|d_{k+1}\|^2 - \|d_k\|^2),\end{aligned} \tag{21}$$

where the $\lambda_q \geqslant 0$ is the maximum eigenvalue of Q. Since the event is not triggered, triggering conditions are not met, then we have

$$\|F(d_k, v_{k_i})\| \leqslant C\|d_k\| + C\|e_k\|. \tag{22}$$

$$\|e(k)\|^2 \leqslant T_e. \tag{23}$$

Substituting (22), (21) is equivalent to

$$\Delta L_k \leqslant \lambda_q((C\|d_k\| + C\|e_k\|)^2 - \|d_k\|^2). \tag{24}$$

In accordance with the Cauthy-Schwarz inequality, we have

$$\Delta L_k \leqslant \lambda_q(2C^2\|d_k\|^2 + 2C^2\|e_k\|^2 - \|d_k\|^2). \tag{25}$$

Combining (25) and (23), we can get $\Delta L_k \leqslant -\lambda_q(\alpha - 2)\|d_k\|^2 \leqslant 0$. That is, $d_{k+1} \leqslant d_k$ is satisfied, when $k \in (k_j, k_{j+1}), j = 0, 1, \ldots, \infty$.

When the event is triggered, i.e. $k = \{k_j\}_{j=0}^{\infty}$, the converted system (7) is asymptotically stable with the policy iteration algorithm, because the converted control are updated to the optimal one [33]. Hence, with the ETCC algorithm, the converted system (7) is stable at the origin, in other words, the actual system (2) can be guaranteed to track the desired trajectory.

The proof is completed. □

Remark 2. Triggering conditions are designed to solve the optimal tracking control problems for nonlinear systems. Different some existing works [45,49], we change a hypothetical condition into a triggering condition (17), and the triggering conditions proposed are easier to obtain without knowing the optimal value function for future moments. Therefore, theoretically, the developed event-triggered algorithm is more feasible.

3.3. Event-triggered actual controller

In this part, the actual controller is designed to be event-triggered. That is, the actual control input u_k will be updated at triggering instants. The architecture block diagram of is presented in Fig. 3

The triggering conditions are chosen by

$$\|e_k\|^2 > T_e, \tag{26}$$

$$\|F(d_k, u_k - \mu_k)\| > T_f. \tag{27}$$

T_e and T_f are the triggering thresholds given by

$$T_e = \frac{\|u_k - \mu_k\|^2 - \|u_k - \mu_{k+1}\|^2}{2C^2\lambda_q}\lambda_r - \|d_k\|^2, \tag{28}$$

$$T_f = C\|d_k\| + C\|e_k\|, \tag{29}$$

where the λ_r is the maximum eigenvalue of R. When at least one of the triggering conditions (26) and (27) is satisfied, the actual control

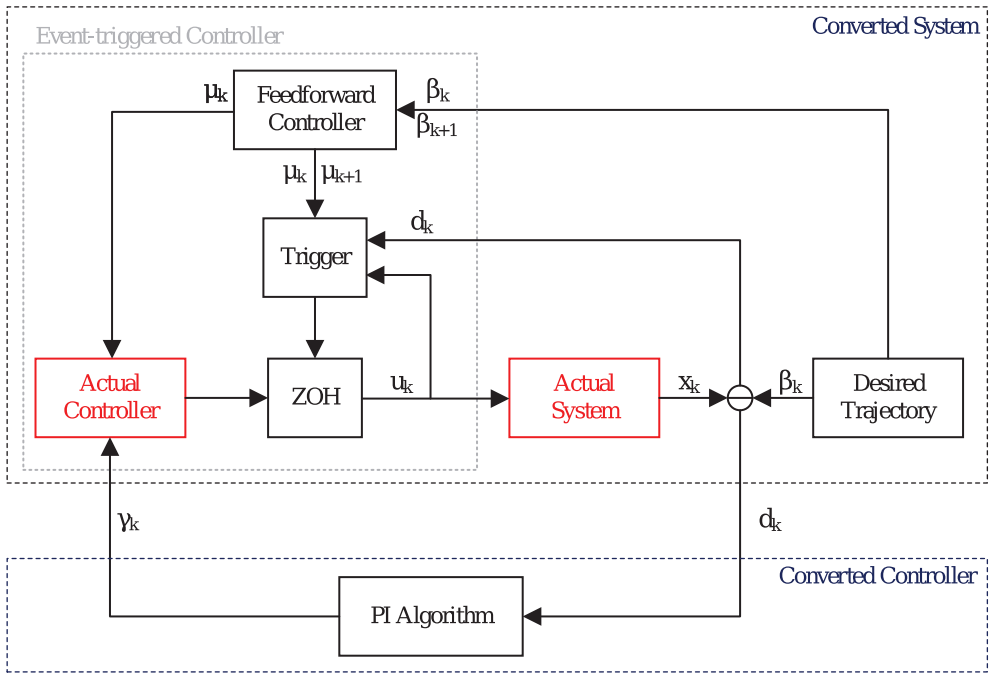

Fig. 3. The event-triggered control algorithm for the actual system.

u_k will be updated, or it will keep constant. The event-triggered control algorithm for the actual system (ETCA) is presented as Algorithm 2,

Algorithm 2: ETCA

Initialization
Let $k = 0; h = 0$;
Take the optimal control policy u_k^* as control input.
1: Compute the triggering error e_k according to (15);
2: Compute the thresholds according to (18) and (19);
3: **if** at least one of (16) and (17) is satisfied**then**
4: Update the actual control input $u_k = u_k^*$.
5: Update the tracking error $d_{k+1} = F(d_k, u_k - \mu_k)$.
6: Set $h = h + 1$.
7: Update the ZOH $u_h = u_k$.
8: Set $k = k + 1$ and goto Step 1. 9: **else**
10: Update the tracking error $d_{k+1} = F(d_k, u_h - \mu_k)$.
11: Set $k = k + 1$, and goto Step 1.

Remark 3. Algorithm 2 makes the actual controller update intermittently, which is significant for saving communication resources. Although the converted controller is still updated in real time with the feedforward controller, the converted controller is a virtual variable and will only take up fewer computational and communication resources.

To make the actual system track the desired trajectory, the stability of the converted system should be guaranteed. In the following, based on the developed ETCA algorithm, the stability analysis of the converted system is presented.

Theorem 2. *Combining the policy iteration algorithm and the event-triggered actual controller, the tracking error of the converted system (7) can be proved to be stable at the origin. That is, the actual system (2) can be guaranteed to track the desired trajectory (1).*

Proof :
Define the Lyapunov function as the form of (20). The first difference of the function is obtained by

$$\begin{aligned}\Delta L_k &= d_{k+1}^T Q d_{k+1} + v_{k+1}^T R v_{k+1} - (d_k^T Q d_k + v_k^T R v_k) \\ &\leqslant \lambda_q(\|d_{k+1}\|^2 - \|d_k\|^2) + \lambda_r(\|v_{k+1}\|^2 - \|v_k\|^2).\end{aligned} \tag{30}$$

When the event is not triggered, the actual controller u_k will not be updated. According to (5), the difference equation can be rewritten as

$$\Delta L_k \leqslant \lambda_q(\|d_{k+1}\|^2 - \|d_k\|^2) + \lambda_r(\|u_k - \mu_{k+1}\|^2 - \|u_k - \mu_k\|^2). \tag{31}$$

Since the triggering conditions are not met, we can obtain

$$\|F(d_k, u_k - \mu_k)\| \leqslant C\|d_k\| + C\|e_k\|. \tag{32}$$

$$\|e(k)\|^2 \leqslant \frac{\|u_k - \mu_k\|^2 - \|u_k - \mu_{k+1}\|^2}{2C^2\lambda_q}\lambda_r - \|d_k\|^2. \tag{33}$$

Substituting (32) into (31), the difference equation can be further derived as

$$\begin{aligned}\Delta L_k \leqslant \lambda_q((C\|d_k\| + C\|e_k\|)^2 - \|d_k\|^2) + \lambda_r(\|u_k - \mu_{k+1}\|^2 \\ - \|u_k - \mu_k\|^2).\end{aligned} \tag{34}$$

Then, according to the Cauthy-Schwarz inequality, it is satisfied that

$$\begin{aligned}\Delta L_k \leqslant \lambda_q(2C^2\|d_k\|^2 + 2C^2\|e_k\|^2 - \|d_k\|^2) + \lambda_r(\|u_k - \mu_{k+1}\|^2 \\ - \|u_k - \mu_k\|^2).\end{aligned} \tag{35}$$

Combining (35) and (33), we can get $\Delta L_k \leqslant -\lambda_q\|d_k\|^2 \leq 0$. That is, $d_{k+1} \leqslant d_k$ is satisfied when $k \in (k_j, k_{j+1}), j = 0, 1, \ldots, \infty$. Besides, when the event is triggered, i.e. $k = \{k_j\}_{j=0}^{\infty}$, the converted system (7) is asymptotically stable with the policy iteration algorithm , because the converted control are updated to the optimal one [33]. Hence, with the ETCA algorithm, the converted system (7) is stable at the origin, in other words, the actual system (2) can be guaranteed to track the desired trajectory.

The proof is completed. □

Remark 4. The optimal tracking control problems are always converted to the optimal regulation problems [48,44]. The converted controller v_k of the optimal regulation system is an intermediate variable defined to facilitate optimization, and the actual controller, which is tuned directly in practice, can be only obtained by (4) and (5). However, the converted controller is mainly researched instead of the actual system in most of existing works [29,44,38,48], including the ETCC in this paper. Thus, for the purpose of facilitating the direct application of the event-triggered ADP algorithm in practice, the ETCA is developed. Additionally, the stability of tracking error at the origin ensures that the actual system can track the desired trajectory.

4. Neural network implementation

In this paper, neural networks are used to implement the algorithms. There are three main parts: the model network, the critic network, and the action network.

The weight matrix between the hidden layer and output layer is denoted as W, and the weight matrix between the input layer and hidden layer is denoted as Y. The activation function is defined as $\sigma(d_k) = (1 - e^{-d_k})/(1 + e^{-d_k})$. ζ is defined as the approximation error of neural networks. The learning rate is denoted as η. The superscripts m, a and c indicates the model network, action network and critic network, respectively. The persistent excitation condition is required during the learning process, that is, a set of data is used as training set for the neural network.

4.1. Model network

Approximate the system (7) with the model network, given by

$$\hat{d}_{k+1} = W_k^{m\top}\sigma(Y_k^{m\top}[d_k; v_k]) + \zeta^m, \tag{36}$$

where W_k^m and Y_k^m denote the weight matrices of the model network. The goal is to minimize the error function as follows,

$$S_k^m = \frac{1}{2}s_k^{m\top}s_k^m, \tag{37}$$

where

$$s_k^m = \hat{d}_k - d_k. \tag{38}$$

The weights update policy for the model network is

$$W_{k+1}^m = W_k^m - \eta_m\left(\frac{\partial S_{k+1}^m}{\partial W_k^m}\right) = W_k^m - \eta^m\left(\frac{\partial S_{k+1}^m}{\partial \hat{d}_{k+1}}\frac{\partial \hat{d}_{k+1}}{\partial W_k^m}\right). \tag{39}$$

$$Y_{k+1}^m = Y_k^m - \eta_m\left(\frac{\partial S_{k+1}^m}{\partial Y_k^m}\right) = Y_k^m - \eta^m\left(\frac{\partial S_{k+1}^m}{\partial \hat{d}_{k+1}}\frac{\partial \hat{d}_{k+1}}{\partial Y_k^m}\right). \tag{40}$$

4.2. Critic network

The critic network is used to approximate the value function $V_i(d_k)$, given by

$$V_i(d_k) = W_i^{c\top}\sigma(Y_i^{c\top}d_k) + \zeta^c, \tag{41}$$

where W_i^c and Y_i^c denote the weight matrices of the critic network. The purpose is to minimize the error function

$$S_k^c = \frac{1}{2}s_k^{c\top}s_k^c, \tag{42}$$

where

$$s_k^c = V_i(d_k) - [V_i(\hat{d}_{k+1}) + U(d_k, v_{k_i})]. \tag{43}$$

The update policy for the critic network satisfies

$$W_{i+1}^c = W_i^c - \eta^c\left(\frac{\partial S_k^c}{\partial W_i^c}\right) = W_i^c - \eta^c\left(\frac{\partial S_k^c}{\partial V_i(d_k)}\frac{\partial V_i(d_k)}{\partial W_i^c}\right). \tag{44}$$

$$Y_{i+1}^c = Y_i^c - \eta^c\left(\frac{\partial S_k^c}{\partial Y_i^c}\right) = Y_i^c - \eta^c\left(\frac{\partial S_k^c}{\partial V_i(d_k)}\frac{\partial V_i(d_k)}{\partial Y_i^c}\right). \tag{45}$$

4.3. Action network

The action network is used to approximate the control input v_{k_i}, given by

$$v_{k_i} = W_i^{a\top}\sigma(Y_i^{a\top}d_k) + \zeta^a, \tag{46}$$

where W_i^a and Y_i^a denote the weight matrices of the action network. The goal is to minimize the following function

$$S_k^a = \frac{1}{2}s_k^{a\top}s_k^a, \tag{47}$$

where

$$s_k^a = V_i(\hat{d}_{k+1}) + U(d_k, v_{k_i}). \tag{48}$$

The weights update policy [50] for the action network satisfies

$$\begin{aligned} W_{i+1}^a &= W_i^a - \eta^a\left(\frac{\partial S_k^a}{\partial W_i^a}\right) \\ &= W_i^a - \eta^a\frac{\partial S_k^a}{\partial V_i(\hat{d}_{k+1})}\frac{\partial V_i(\hat{d}_{k+1})}{\partial \hat{d}_{k+1}}\frac{\partial \hat{d}_{k+1}}{\partial v_{k_i}}\frac{\partial v_{k_i}}{\partial W_i^a} - \eta^a\frac{\partial S_k^a}{\partial U(d_k, v_{k_i})}\frac{\partial U(d_k, v_{k_i})}{\partial v_{k_i}}\frac{\partial v_{k_i}}{\partial W_i^a}. \end{aligned} \tag{49}$$

$$\begin{aligned} Y_{i+1}^a &= Y_i^a - \eta^a\left(\frac{\partial S_k^a}{\partial Y_i^a}\right) \\ &= Y_i^a - \eta^a\frac{\partial S_k^a}{\partial V_i(\hat{d}_{k+1})}\frac{\partial V_i(\hat{d}_{k+1})}{\partial \hat{d}_{k+1}}\frac{\partial \hat{d}_{k+1}}{\partial v_{k_i}}\frac{\partial v_{k_i}}{\partial Y_i^a} - \eta^a\frac{\partial S_k^a}{\partial U(d_k, v_{k_i})}\frac{\partial U(d_k, v_{k_i})}{\partial v_{k_i}}\frac{\partial v_{k_i}}{\partial Y_i^a}. \end{aligned} \tag{50}$$

5. Examples

Two simulations are employed to show the effectiveness of the methods studied.

Example.1 Considering the 2-DOF helicopter system in [51], a state-space description of the helicopter is given as follows,

$$\dot{x} = ax + bu, \tag{51}$$

where $x = [\theta, \psi, \dot{\theta}, \dot{\psi}]^\top \in \mathbb{R}^4$, $u = [Fp, Fy]^\top \in \mathbb{R}^2$. θ denotes the pitch angle, ψ denotes the yaw angle, Fp is the controller on the pitch axis, and Fy is the controller on the yaw axis. a and b are given as

$$a = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -B_p/J_{Tp} & 0 \\ 0 & 0 & 0 & -B_y/J_{Ty} \end{bmatrix}, \tag{52}$$

$$b = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ K_{pp}/J_{Tp} & K_{py}/J_{Tp} \\ K_{yp}/J_{Ty} & K_{yy}/J_{Ty} \end{bmatrix}. \tag{53}$$

The meaning of the parameters is given in Table 1.

Then a computable dynamic model is obtained by discretize the dynamic equations with sampling time $T = 0.1$ sec,

$$\begin{bmatrix} \theta_{k+1} \\ \psi_{k+1} \\ \theta_{k+2} \\ \psi_{k+2} \end{bmatrix} = A\begin{bmatrix} \theta_k \\ \psi_k \\ \theta_{k+1} \\ \psi_{k+1} \end{bmatrix} + B\begin{bmatrix} Fp_k \\ Fy_k \end{bmatrix}, \tag{54}$$

Table 1
Notations [51].

Parameters	Meaning
J_p	total moment of inertia about pitch axis
J_y	total moment of inertia about yaw axis
m	total mass of the helicopter
B_p	equivalent viscous damping about pitch axis
B_y	equivalent viscous damping about yaw axis
K_{pp}	thrust force constant of yaw propeller
K_{yy}	thrust torque constant acting on yaw axis from yaw propeller
K_{py}	thrust torque constant acting on pitch axis from yaw propeller
K_{yp}	thrust torque constant acting on yaw axis from pitch propeller

where A and B are written as

$$A = \begin{bmatrix} 1 & 0 & 0.1 & 0 \\ 0 & 1 & 0 & 0.1 \\ 0 & 0 & 1-0.1B_p/J_{Tp} & 0 \\ 0 & 0 & 0 & 1-0.1B_y/J_{Ty} \end{bmatrix}, \tag{55}$$

$$B = 0.1\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ K_{pp}/J_{Tp} & K_{py}/J_{Tp} \\ K_{yp}/J_{Ty} & K_{yy}/J_{Ty} \end{bmatrix}. \tag{56}$$

Define the desired trajectory as

$$\beta_k = \begin{bmatrix} \sin^2(0.5k) \\ \sin(0.5k) \\ 0.5\sin(k) \\ 0.5\cos(0.5k) \end{bmatrix}. \tag{57}$$

The performance index function is defined as

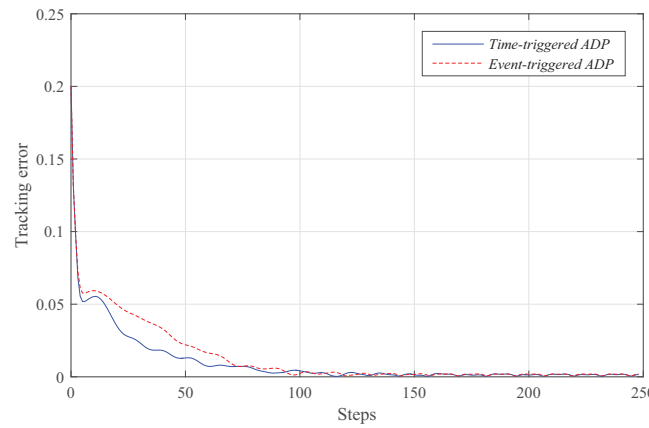

Fig. 5. The Euclidean norm of tracking errors.

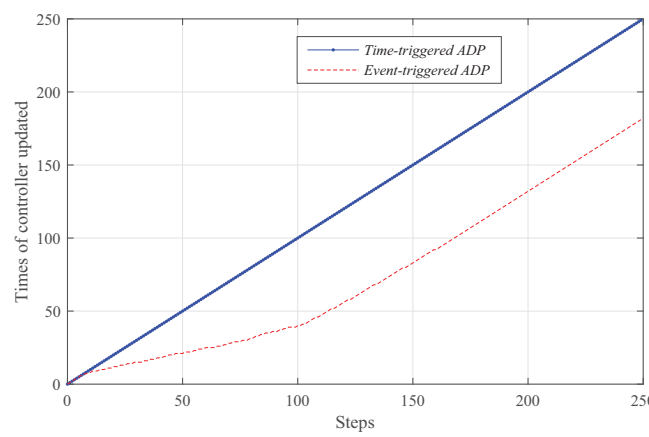

Fig. 6. Comparison of training times.

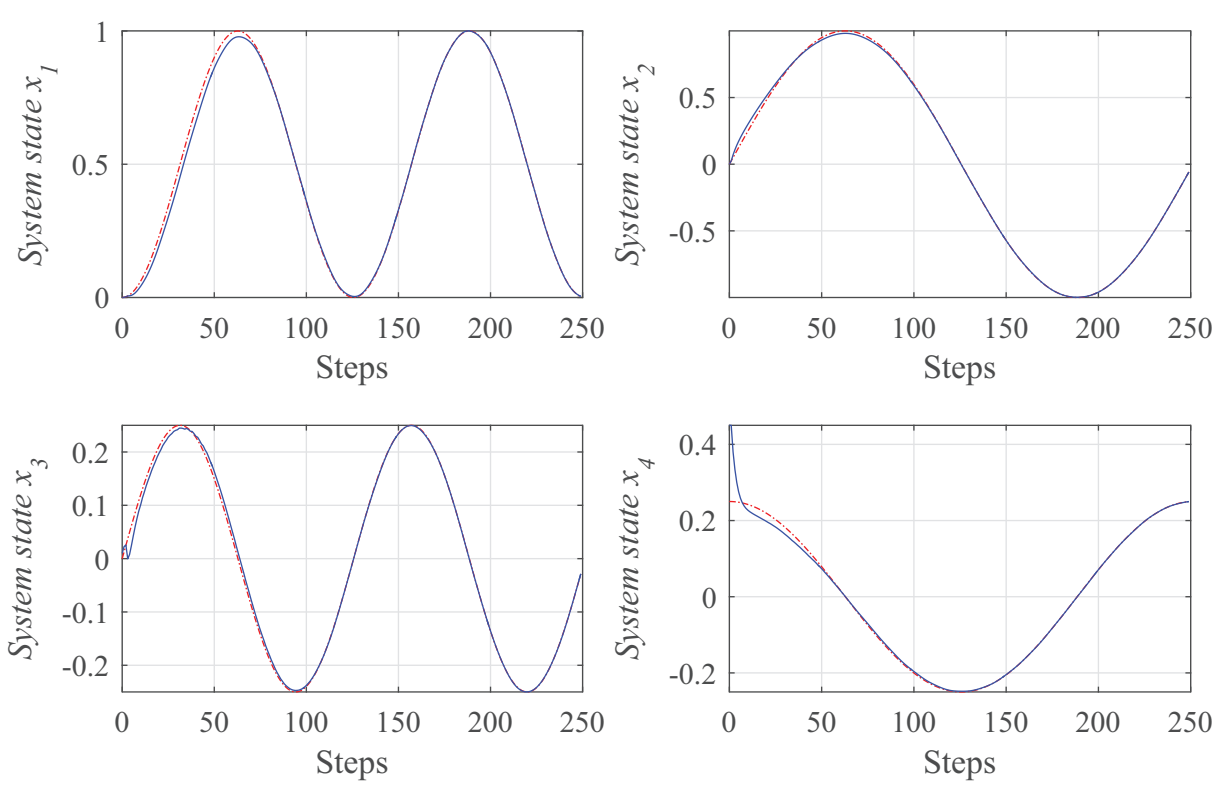

Fig. 4. The tracking trajectories based on the ETCC algorithm.

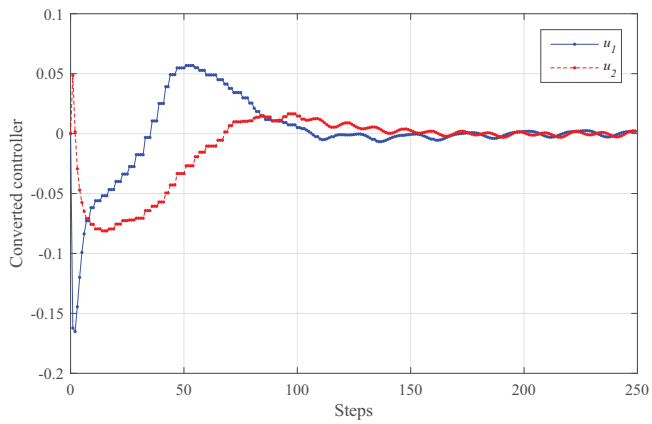

Fig. 7. The converted controller.

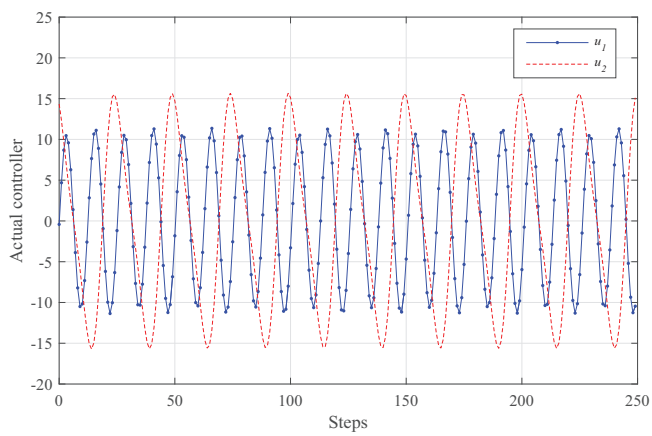

Fig. 8. The actual controller.

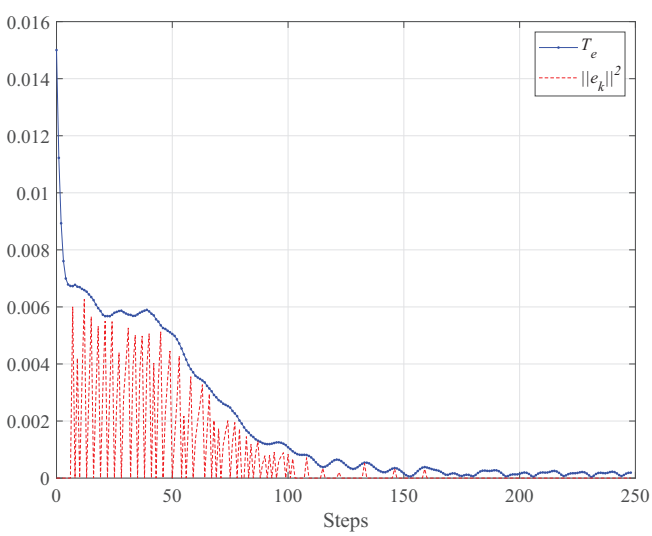

Fig. 9. The triggering condition (26).

$$J = \sum_{k=0}^{\infty} (d_k^{\mathrm{T}} Q d_k + v_k^{\mathrm{T}} R v_k), \tag{58}$$

where $Q = I, R = I$, and I is the identity matrix with appropriate dimensions.

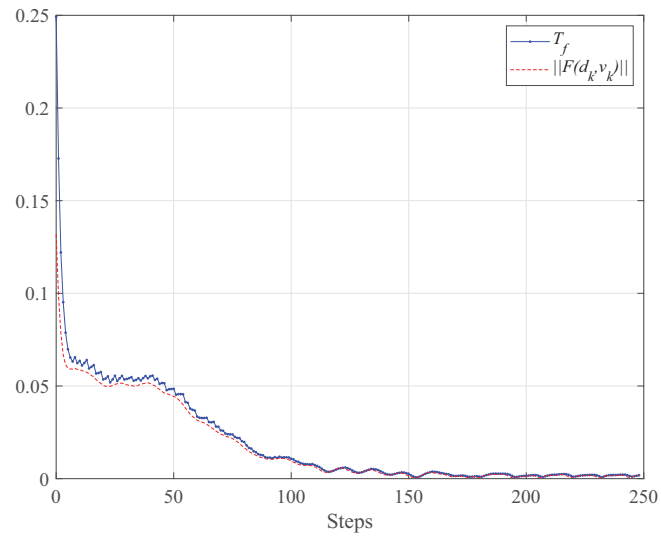

Fig. 10. The triggering condition (27).

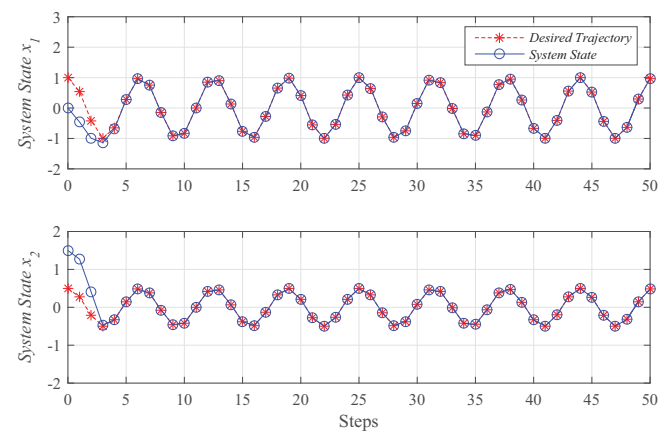

Fig. 11. The tracking trajectories based on ETCA.

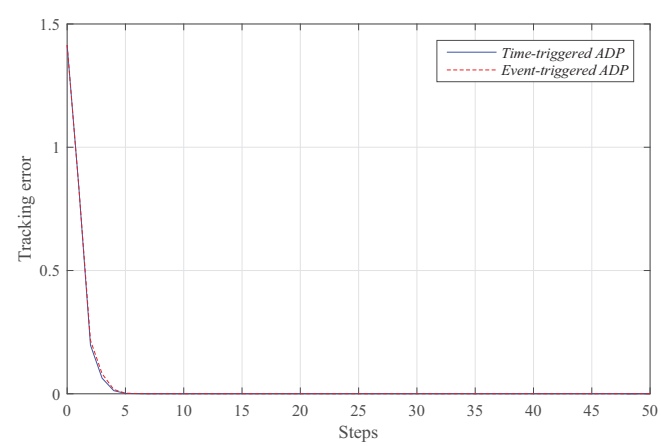

Fig. 12. The Euclidean norm of tracking errors.

The actual controller is designed to make states track the desired trajectories (57). We compare the time-triggered ADP algorithm and the ETCC algorithm to explore performance differences. BP neural networks with the structures of 6–8–4, 4–8–2 and 4–8–1 are employed to implement the algorithms. The learning rates of

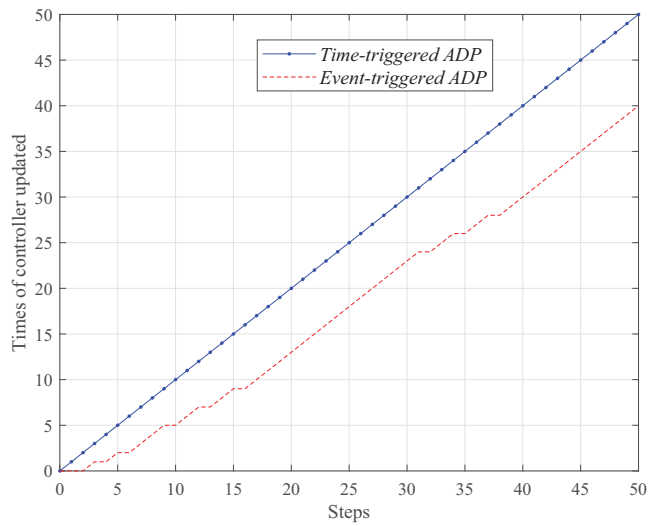

Fig. 13. Comparison of update times.

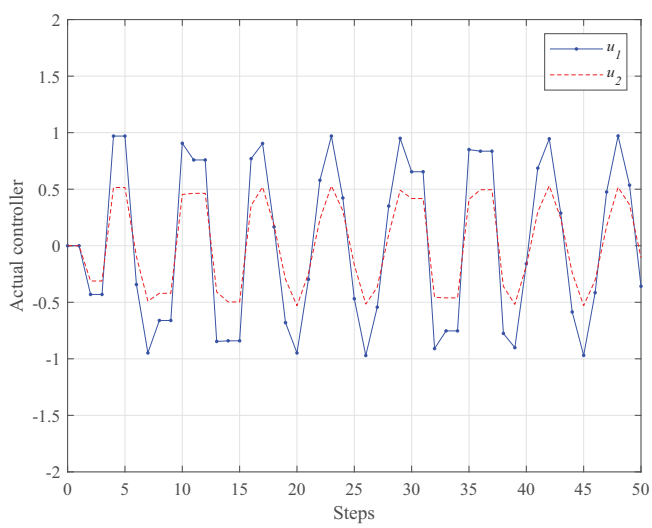

Fig. 14. The control law of the actual system.

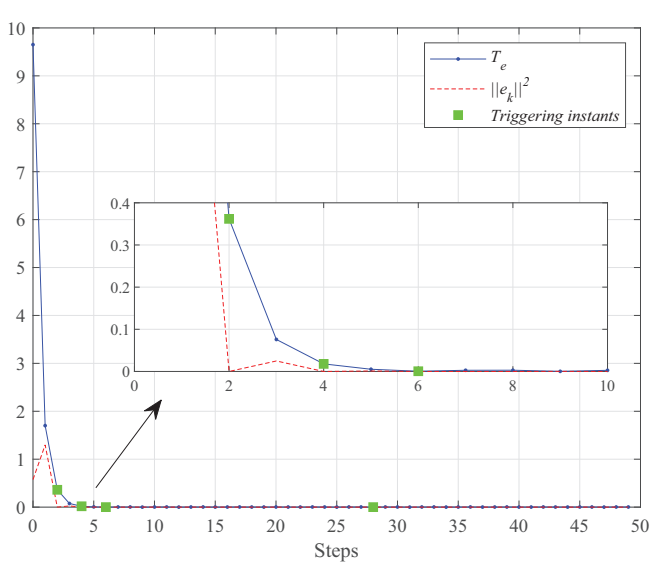

Fig. 15. The triggering condition (16).

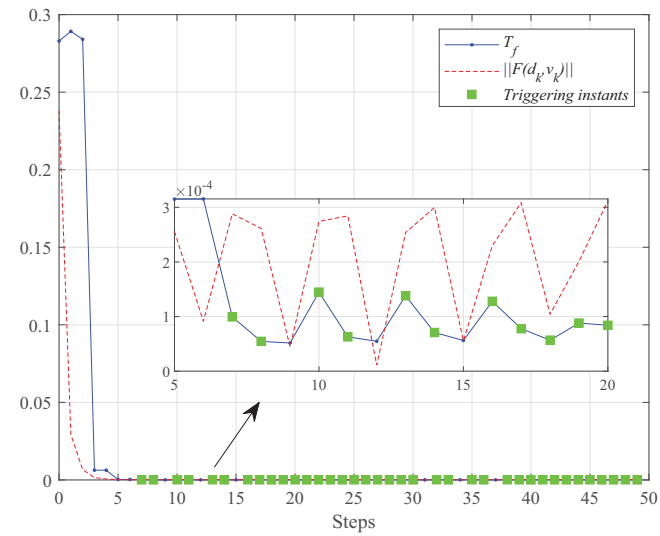

Fig. 16. The triggering condition (17).

the critic network and the action network are chosen as 0.03. In each iteration, the neural network will be trained for 1000 times. The initial tracking error is given as $d_0 = [0, 0, 0, 0.25]$.

In Fig. 4, the trajectories based on the ETCC algorithm are shown. To compare with the time-triggered ADP method, the Euclidean norm of tracking errors of the two methods are presented in Fig. 5. The tracking errors are both close to zero at the 100th sampling point, that is, the tracking performance of the two methods is comparable. The times of the actual controller updated are counted in Fig. 6. The time-triggered algorithm is completed by 250 times update, and the event-triggered algorithm by 172, which indicates a 31% reduction in the number of updates.

The converted controller is presented in Fig. 7, and the actual controller is shown in Fig. 8. Obviously, with the ETCC algorithm, only the converted controller is event-triggered, while the actual controller is updated continuously.

The square of the triggering error $\|e_k\|^2$ and the triggering threshold T_e are shown in Fig. 9. According to the triggering condition (26), once $\|e_k\|^2$ is greater than T_e, the event will be triggered, and the triggering error will be reset. According to the triggering condition (27), the trajectories of the prediction $\|F(d_k, v_k)\|$ and T_f are shown in Fig. 10. When $\|F(d_k, v_k)\|$ is greater than T_f, the event will be triggered. In summary, comparing with the time-triggered ADP algorithm, one can obtain comparable performance with less computation under the ETCC algorithm.

Example.2 Consider the example in [48] with some modifications, an affine nonlinear system is given as follows:

$$x_{k+1} = f(x_k) + gu_k, \tag{59}$$

where $x_k \in \mathbb{R}^2, u_k \in \mathbb{R}^2, f(x_k)$ and g are written as

$$f(x_k) = \begin{bmatrix} 0.2x_{1k}e^{x_{2k}^2} \\ 0.3x_{2k}^3 \end{bmatrix}, \tag{60}$$

$$g = \begin{bmatrix} 0.9 & 0 \\ 0 & 0.9 \end{bmatrix}. \tag{61}$$

Define the desired trajectory as

$$\beta_k = [\sin(k + \pi/2) \quad 0.5\cos(k)]. \tag{62}$$

Define the performance index function as

$$J = \sum_{k=0}^{\infty}(d_k^{\mathrm{T}}Qd_k + v_k^{\mathrm{T}}Rv_k), \tag{63}$$

where $Q = I, R = I$, and I is the identity matrix with appropriate dimensions. Implement time-triggered ADP algorithm and ETCA algorithm for comparison. Neural networks with the structures of 4–8–2, 2–8–2 and 2–8–1 are employed. The learning rates of the neural networks are chosen as 0.1, and each network is trained for 1000 times per iteration. The initial tracking error is given as $d_0 = [-1, 1]$.

The trajectories based on the ETCA algorithm are presented in Fig. 11. To facilitate the comparison, we also test the time-triggered ADP algorithm, and the Euclidean norm of tracking errors based on these two methods is depicted in Fig. 12. It can be seen that the tracking error is close to zero at the 5th sampling point, that is, the tracking performance of the two methods is comparable.

The times of controller updated based on the two methods are counted in Fig. 13. The time-triggered ADP algorithm is completed by 50 times update, while the event-triggered algorithm by 42. And the actual controller is shown in Fig. 14. Apparently, the actual controller is event-triggered. In short, Figs. 13 and 14 indicate that the communication requirements for the actual controller is reduced.

The square of the triggering error $\|e_k\|^2$ and the triggering threshold T_e are shown in Fig. 15. According to the triggering condition (16), once $\|e_k\|^2$ is greater than T_e, the event will be triggered, and the triggering error will be reset. According to the triggering condition (17), the trajectories of prediction $\|F(d_k, v_k)\|$ and T_f are shown in Fig. 16. When $\|F(d_k, v_k)\|$ is greater than T_f, the event will be triggered.

In summary, comparing with the time-triggered ADP algorithm, we can get comparable control performance with less computation based on the ETCA algorithm.

6. Conclusion

In this paper, the event-triggered scheme is studied to reduce the communication demand for solving optimal tracking control problems. Two event-triggered algorithms based on ADP algorithms, ETCC and ETCA, are developed with novel triggering conditions. Compared with existing works, the developed algorithms are more feasible due to the lack of a hypothetical condition, and the triggering conditions proposed are easier to obtain without knowing the optimal value function for future moments. Besides, the ETCA algorithm designed for the actual system facilitates the direct application in practice. Additionally, the stability at the origin of the converted system is studied based on the proposed algorithms, which proves that the actual system can track the desired trajectory accurately. Finally, two examples are employed to illustrate the effectiveness of the two algorithms, respectively. In summary, the event-triggered ADP methods could reduce the communication costs and maintain comparable control performance.

In future work, we will study the following topics. First, we are going to compare different event-triggered optimal controllers for discrete-time optimal tracking control problems. Second, we will focus on the performance guaranteed event-triggered ADP algorithms for discrete-time optimal tracking control problems. Third, We would like to explore the event-triggered online ADP algorithms.

CRediT authorship contribution statement

Ziyang Wang: Conceptualization, Methodology, Software, Validation, Investigation, Writing - original draft. **Joonhyup Lee:** Investigation, Writing - review & editing. **Qinglai Wei:** Conceptualization, Writing - review & editing. **Anting Zhang:** Investigation, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Haoyan Zhang, Huanqing Wang, Ben Niu, Liang Zhang, Adil M. Ahmad, Sliding-mode surface-based adaptive actor-critic optimal control for switched nonlinear systems with average dwell time, Inf. Sci. 580 (2021) 756–774.
[2] Shanlin Liu, Ben Niu, Guangdeng Zong, Xudong Zhao, Xu. Ning, Data-driven-based event-triggered optimal control of unknown nonlinear systems with input constraints, Nonlinear Dyn. 109 (2022) 891–909.
[3] Karl J. Aström, Event Based Control, Springer, Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 127–147.
[4] M. Mazo, P. Tabuada, Decentralized event-triggered control over wireless sensor/actuator networks, IEEE Trans. Autom. Control 56 (10) (2011) 2456–2461.
[5] Yang Liu, Bangxin Jiang, Lu. Jianquan, Jinde Cao, Lu. Guoping, Event-triggered sliding mode control for attitude stabilization of a rigid spacecraft, IEEE Trans. Syst., Man, Cybern.: Syst. 50 (9) (2020) 3290–3299.
[6] Du. Sheng-Li, Tao Liu, Daniel W.C. Ho, Dynamic event-triggered control for leader-following consensus of multiagent systems, IEEE Trans. Syst., Man, Cybern.: Syst. 50 (9) (2020) 3243–3251.
[7] P. Tallapragada, N. Chopra, On event triggered tracking for nonlinear systems, IEEE Trans. Autom. Control 58 (9) (2013) 2343–2348.
[8] Antoine Girard, Dynamic triggering mechanisms for event-triggered control, IEEE Trans. Autom. Control 60 (7) (2015) 1992–1997.
[9] Nowzaria Cameron, Garciab Eloy, Cortcs Jorge, Event-triggered communication and control of networked systems for multi-agent consensus, Automatica 105 (2019) 1–27.
[10] Xianwei Li, Zhiyong Sun, Yang Tang, Hamid Reza Karimi, Adaptive event-triggered consensus of multiagent systems on directed graphs, IEEE Trans. Autom. Control 66 (4) (2021) 1670–1685.
[11] V.S. Dolk, D.P. Borgers, W.P.M.H. Heemels, Output-based and decentralized dynamic event-triggered control with guaranteed $\mathscr{L}_p$- gain performance and zeno-freeness, IEEE Trans. Autom. Control 62 (1) (2017) 34–49.
[12] Xian-Ming Zhang, Qing-Long Han, Bao-Lin Zhang, An overview and deep investigation on sampled-data-based event-triggered control and filtering for networked systems, IEEE Trans. Industr. Inf. 13 (1) (2017) 4–16.
[13] D.P. Borgers, W.P.M.H. Heemels, Event-separation properties of event-triggered control systems, IEEE Trans. Autom. Control 59 (10) (2014) 2644–2656.
[14] M.C.F. Donkers, W.P.M.H. Heemels, Output-based event-triggered control with guaranteed L_∞-gain and improved and decentralized event-triggering, IEEE Trans. Autom. Control 57 (6) (2012) 1362–1376.
[15] Romain Postoyan, Paulo Tabuada, Dragan Nešić, Adolfo Anta, A framework for the event-triggered stabilization of nonlinear systems, IEEE Trans. Autom. Control 60 (4) (2015) 982–996.
[16] Anton Selivanov, Emilia Fridman, Event-triggered h_∞ control: A switching approach, IEEE Trans. Autom. Control 61 (10) (2016) 3221–3226.
[17] Biao Luo, Yin Yang, Derong Liu, Wu. Huai-Ning, Event-triggered optimal control with performance guarantees using adaptive dynamic programming, IEEE Trans. Neural Networks Learn. Syst. 31 (1) (2020) 76–88.
[18] Xiangnan Zhong, Haibo He, Grhdp solution for optimal consensus control of multiagent discrete-time systems, IEEE Trans. Syst., Man, Cybern.: Syst. 50 (7) (2020) 2362–2374.
[19] Yi Chang, Peng Zhou, Ben Niu, Huanqing Wang, Xu. Ning, M.O. Alassafi, A.M. Ahmad, Switched-observer-based adaptive output-feedback control design with unknown gain for pure-feedback switched nonlinear systems via average dwell time, Int. J. Syst. Sci. 52 (9) (2021) 1731–1745.
[20] Xiaoheng Chang, Yuanwei Jing, Xiying Gao, Xiaoping Liu, h_∞ tracking control design of t-s fuzzy systems, Control Decision 23 (3) (2008) 329–332.
[21] P.J. Werbos, Advanced forecasting methods for global crisis warning and models of intelligence, General Syst. Yearbook 22 (6) (1977) 25–38.
[22] J.J. Murray, C.J. Cox, G.G. Lendaris, R. Saeks, Adaptive dynamic programming, IEEE Trans. Syst., Man, Cybern., Part C (Appl. Rev.) 32 (2) (2002) 140–153.
[23] A. Al-Tamimi, F.L. Lewis, M. Abu-Khalaf, Discrete-time nonlinear HJB solution using approximate dynamic programming: Convergence proof, IEEE Trans. Syst., Man, Cybern., Part B (Cybern.) 38 (4) (2008) 943–949.
[24] D. Liu, Q. Wei, W. Ding, Y. Xiong, H. Li, Adaptive Dynamic Programming with Applications in Optimal Control, Springer International Publishing, 2017.
[25] B. Luo, D. Liu, T. Huang, J. Liu, Output tracking control based on adaptive dynamic programming with multistep policy evaluation, IEEE Trans. Syst., Man, Cybern.: Syst. 49 (10) (2019) 2155–2165.

[26] D. Liu, S. Xue, B. Zhao, B. Luo, Q. Wei, Adaptive dynamic programming for control: A survey and recent advances, IEEE Trans. Syst., Man, Cybern.: Syst. 51 (1) (2021) 142–160.
[27] T. Dierks and S. Jagannathan. Optimal tracking control of affine nonlinear discrete-time systems with unknown internal dynamics. In IEEE Conference on Decision and Control, 2010.
[28] B. Zhao, D. Liu, Event-triggered decentralized tracking control of modular reconfigurable robots through adaptive dynamic programming, IEEE Trans. Industr. Electron. 67 (4) (2020) 3054–3064.
[29] Lu. Jingwei, Qinglai Wei, Fei-Yue Wang, Parallel control for optimal tracking via adaptive dynamic programming, IEEE/CAA J. Automatica Sinica 7 (6) (2020) 1662–1674.
[30] Bahare Kiumarsi, Kyriakos G. Vamvoudakis, Hamidreza Modares, Frank L. Lewis, Optimal and autonomous control using reinforcement learning: A survey, IEEE Trans. Neural Networks Learn. Syst. 29 (6) (2018) 2042–2062.
[31] Qinglai Wei, Ruizhuo Song, Zehua Liao, Benkai Li, Frank L. Lewis, Discrete-time impulsive adaptive dynamic programming, IEEE Trans. Cybern. 50 (10) (2020) 4293–4306.
[32] Y. Yang, H. Modares, D.C. Wunsch, Y. Yin, Leader-follower output synchronization of linear heterogeneous systems with active leader using reinforcement learning, IEEE Trans. Neural Networks Learn. Syst. 29 (6) (2018) 2139–2153.
[33] Derong Liu, Qinglai Wei, Policy iteration adaptive dynamic programming algorithm for discrete-time nonlinear systems, IEEE Trans. Neural Networks Learn. Syst. 25 (3) (2014) 621–634.
[34] Vahidreza Nasirian, Qobad Shafiee, Josep M. Guerrero, Frank L. Lewis, Ali Davoudi, Droop-free distributed control for ac microgrids, IEEE Trans. Power Electron. 31 (2) (2016) 1600–1617.
[35] D. Liu, Y. Xu, Q. Wei, X. Liu, Residential energy scheduling for variable weather solar energy based on adaptive dynamic programming, IEEE/CAA J. Autom. Sinica 5 (1) (2018) 36–46.
[36] Hao Liu, Yanhu Wang, Frank L. Lewis, Robust distributed formation controller design for a group of unmanned underwater vehicles, IEEE Trans. Syst., Man, Cybern.: Syst. 51 (2) (2021) 1215–1223.
[37] Huaguang Zhang, Lili Cui, Xin Zhang, Yanhong Luo, Data-driven robust approximate optimal tracking control for unknown general nonlinear systems using adaptive dynamic programming method, IEEE Trans. Neural Networks 22 (12) (2011) 2226–2236.
[38] Mu. Chaoxu, Zhen Ni, Changyin Sun, Haibo He, Air-breathing hypersonic vehicle tracking control based on adaptive dynamic programming, IEEE Trans. Neural Networks Learn. Syst. 28 (3) (2017) 584–598.
[39] Bo Zhao, Derong Liu, Cesare Alippi, Sliding-mode surface-based approximate optimal control for uncertain nonlinear systems with asymptotically stable critic structure, IEEE Trans. Cybern. 51 (6) (2021) 2858–2869.
[40] Avimanyu Sahoo, Vignesh Narayanan, Sarangapani Jagannathan, A min-max approach to event- and self-triggered sampling and regulation of linear systems, IEEE Trans. Industr. Electron. 66 (7) (2019) 5433–5440.
[41] Ziyang Wang, Qinglai Wei, Derong Liu, A novel triggering condition of event-triggered control based on heuristic dynamic programming for discrete-time systems, Optimal Control Appl. Methods 39 (3) (2018) 1467–1478.
[42] Ziyang Wang, Qinglai Wei, Derong Liu, Event-triggered adaptive dynamic programming for discrete-time multi-player games, Inf. Sci. 506 (2020) 457–470.
[43] Ding Wang, Derong Liu, Learning and guaranteed cost control with event-based adaptive critic implementation, IEEE Trans. Neural Networks Learn. Syst. 29 (12) (2018) 6004–6014.
[44] Ziyang Wang, Qinglai Wei, and Derong Liu. A tracking control method based on event-triggered adaptive dynamic programming. In 2019 Chinese Control Conference (CCC), 2019, pp. 2454–2459.
[45] Lu. Jingwei, Qinglai Wei, Yujia Liu, Tianmin Zhou, Fei-Yue Wang, Event-triggered optimal parallel tracking control for discrete-time nonlinear systems, IEEE Trans. Syst., Man, Cybern.: Syst. 52 (6) (2022) 3772–3784.
[46] Yuan-Xin Li, Guang-Hong Yang, Observer-based fuzzy adaptive event-triggered control codesign for a class of uncertain nonlinear systems, IEEE Trans. Fuzzy Syst. 26 (3) (2018) 1589–1599.
[47] Zhan Shi, Zhanshan Wang, Adaptive output-feedback optimal control for continuous-time linear systems based on adaptive dynamic programming approach, Neurocomputing 438 (2021) 334–344.
[48] Huaguang Zhang, Qinglai Wei, Yanhong Luo, A novel infinite-time optimal tracking control scheme for a class of discrete-time nonlinear systems via the greedy hdp iteration algorithm, IEEE Trans. Syst., Man, Cybern., Part B (Cybern.) 38 (4) (2008) 937–942.
[49] Lu. Dong, Xiangnan Zhong, Changyin Sun, Haibo He, Adaptive event-triggered control based on heuristic dynamic programming for nonlinear discrete-time systems, IEEE Trans. Neural Networks Learn. Syst. 28 (7) (2017) 1594–1605.
[50] J. Si, Yu-Tsung Wang, Online learning control by association and reinforcement, IEEE Trans. Neural Networks 12 (2) (2001) 264–276.
[51] Weinan Gao, Mengzhe Huang, Zhong-Ping Jiang, Tianyou Chai, Sampled-data-based adaptive optimal output-feedback control of a 2-degree-of-freedom helicopter, IET Control Theory Appl. 10 (12) (2016) 1440–1447.

Ziyang Wang received the B.S. degree in automation, and Ph.D. degree in Control Science and Engineering, all from the University of Science and Technology Beijing, Beijing, China, in 2014 and 2020, respectively. From 2020 to 2022, he was a Postdoctoral Fellow with the Department of Automation, Tsinghua University, Beijing, China. He is currently an Assistant Professor with Xilingol Vocational College. His research interests include: adaptive dynamic programming, group decision-making, optimal control and crowd intelligence science.

Joonhyup Lee received the B.S. degree in automation, and M.S degree in Control Science and Engineering, all from Tsinghua University, Beijing, China, in 2019 and 2022, respectively. His research interests include: machine learning and group decision-making.

Qinglai Wei received the B.S. degree in automation and the Ph.D. degree in control theory and control engineering from Northeastern University, Shenyang, China, in 2002 and 2009, respectively. From 2009 to 2011, he was a Postdoctoral Fellow with the State Key Laboratory of Management and Control for Complex Systems, Institute of Automation, Chinese Academy of Sciences, Beijing, China, where he is currently a Professor and the Associate Director. He has authored four books, and published over 80 international journal papers. His research interests include adaptive dynamic programming, neural-networks-based control, optimal control, nonlinear systems, and their industrial applications.

Anting Zhang associate professor, graduated from Tsinghua University in 1996 with a master's degree in automatic control theory and application. From 1996 to 2022, he was engaged in scientific research in the Department of Automation of Tsinghua University. His main research direction is enterprise informatization, e-commerce, big data, and crowd science.